

Tools in Google ADK

Agent Development Kit — How Tools Are Defined and Used

1. What is a Tool in ADK?

Google ADK (Agent Development Kit) is Google's open-source framework for building AI agents using Gemini models. In ADK, a Tool is a programming function that an agent can call to take action or retrieve information that is outside the model's own knowledge.

The LLM inside the agent does not execute tools itself. It reads the available tools, decides which one to call and with what inputs, then ADK executes the function and passes the result back to the model. The model then uses that result to decide its next step.

One-liner: A tool is a Python (or TypeScript/Java/Go) function that you define. ADK makes it available to the agent's LLM, which calls it autonomously when needed.

How the agent uses a tool — the flow

- Agent receives a user message and reads its list of available tools
- LLM analyses the message and decides which tool, if any, to call
- LLM generates the input arguments for that tool
- ADK executes the function and captures the return value
- Return value is fed back to the LLM as an observation
- LLM uses the result to continue reasoning and formulate its next action or final response

Key rule: The LLM decides WHEN to call a tool based on the tool's name and docstring. Write clear, descriptive docstrings — they are not just comments, they are the LLM's instruction manual for the tool.

2. The Five Tool Types in ADK

ADK organizes tools into five categories. Each one is suited to a different situation.

Tool Type	What it is	When to use it
Function Tool	A regular Python/TS/Java/Go function you write	Any custom logic — API calls, calculations, lookups
Long-Running Function Tool	An async function for slow operations	File processing, external jobs, polling tasks
Agent-as-Tool	Another agent used as a callable tool	Delegating sub-tasks to a specialist agent
Built-in Tool	Pre-made tools from Google (Search, Code Exec, RAG)	Common tasks — no code needed
MCP / OpenAPI Tool	Tools from MCP servers or REST APIs via spec file	Connecting to external services and enterprise APIs

3. Function Tools — The Most Common Type

Function tools are the ones you will write most often. You define a regular function, wrap it in `FunctionTool`, and pass it to the agent. That is it.

How to define a Function Tool (Python)

The three things that matter most:

- Function name — the LLM reads this to understand what the tool does
- Type hints on all parameters — ADK uses these to build the schema the LLM receives
- Docstring — describes what the tool does, what each parameter means, and what it returns. The LLM reads this to decide when to call it.

```
from google.adk.agents import Agent
from google.adk.tools import FunctionTool

def get_product_price(product_id: str, currency: str = 'USD') -> dict:
    """
    Retrieves the current price of a product from the catalogue.

    Args:
        product_id: The unique ID of the product to look up.
        currency: The currency code to return the price in. Default is USD.

    Returns:
        A dict with 'price' (float) and 'currency' (str), or 'error' if not found.
    """
    # your logic here
    return {'price': 29.99, 'currency': currency}

price_tool = FunctionTool(func=get_product_price)

agent = Agent(
    model='gemini-2.0-flash',
    name='shopping_agent',
    instruction='Help users find product prices. Use get_product_price when asked.',
    tools=[price_tool]
)
```

TypeScript version

```
import { LlmAgent, FunctionTool } from '@google/adk';

const priceTool = new FunctionTool({
  name: 'get_product_price',
  description: 'Retrieves the current price of a product from the catalogue.',
  parameters: { product_id: { type: 'string' }, currency: { type: 'string' } },
  execute: async ({ product_id, currency = 'USD' }) => {
    return { price: 29.99, currency };
  }
});
```

Important: In Python, ADK auto-generates the tool schema from type hints and the docstring — you do not write JSON schema manually. In TypeScript, you define the parameters object explicitly.

Long-Running Function Tools

For tools that take time — like polling an external job, processing a large file, or waiting for a webhook — ADK provides `LongRunningFunctionTool`. It lets the agent continue reasoning while the tool runs in the background and then receives the result when it is ready.

```

from google.adk.tools import LongRunningFunctionTool

async def run_batch_analysis(dataset_id: str) -> dict:
    """Runs a long batch analysis job. Returns job_id immediately, result when done."""
    # kick off async job, return interim result
    return {'status': 'running', 'job_id': 'abc123'}

batch_tool = LongRunningFunctionTool(func=run_batch_analysis)

```

4. ToolContext — Giving Tools More Power

By default, a tool just takes inputs and returns outputs. But if you need a tool to read session state, trigger agent transfers, handle file artifacts, or deal with auth — you use ToolContext.

To get it, just add `tool_context: ToolContext` as a parameter to your function. ADK injects it automatically at runtime. Do not include it in the docstring — the LLM does not need to know about it.

What ToolContext gives you

Attribute / Method	What it does
<code>tool_context.state</code>	Read and write to the current session's state (like a persistent dict across agent steps)
<code>tool_context.actions.transfer_to_agent</code>	Set this to an agent name to hand off control to a different agent mid-conversation
<code>tool_context.actions.escalate</code>	Signals the agent to stop and escalate to a human
<code>tool_context.load_artifact(name)</code>	Load a file (PDF, image, doc) that was uploaded to the session
<code>tool_context.save_artifact(name, data)</code>	Save a new file (e.g. generated report) to the session's artifact store
<code>tool_context.auth_response</code>	Access OAuth tokens or credentials if the tool uses authenticated APIs
<code>tool_context.function_call_id</code>	Unique ID for this tool invocation — useful for tracing and debugging

Example — reading and writing session state

```

from google.adk.tools import FunctionTool, ToolContext

def save_user_language(language: str, tool_context: ToolContext) -> dict:
    """Saves the user preferred language to their session profile."""
    tool_context.state['user:language'] = language
    return {'status': 'saved', 'language': language}

lang_tool = FunctionTool(func=save_user_language)

```

Example — transferring to another agent

```

def check_urgency(query: str, tool_context: ToolContext) -> str:
    """Checks if the query needs urgent support. Transfers if so."""
    if 'urgent' in query.lower() or 'emergency' in query.lower():
        tool_context.actions.transfer_to_agent = 'priority_support_agent'
        return 'Transferring to priority support.'
    return 'Query is standard priority.'

```

State key convention: ADK uses prefixed keys in state: 'user:key' for per-user data, 'session:key' for per-session data, 'app:key' for app-wide data. Stick to this pattern.

5. Built-in Tools

ADK ships with a small set of ready-to-use tools. You do not write any logic — just import and attach them to the agent.

Built-in Tool	What it does	How to use
google_search	Grounds the agent's responses in live Google Search results	<code>from google.adk.tools import google_search</code>
code_execution	Lets the agent write and run Python code in a sandboxed environment	<code>from google.adk.tools import code_execution</code>
RAG Engine tool	Connects the agent to a Vertex AI RAG corpus for document retrieval	via <code>vertex_ai_rag_retrieval</code> tool

```
from google.adk.agents import Agent
from google.adk.tools import google_search, code_execution

analyst_agent = Agent(
    model='gemini-2.0-flash',
    name='analyst',
    instruction='Answer questions using live search. Run code when you need calculations.',
    tools=[google_search, code_execution]
)
```

6. Agent-as-Tool

One of the most powerful features in ADK. You can take any agent and use it as a tool for a parent (orchestrator) agent. The parent calls it like a function and gets back the sub-agent's final response.

This is the main building block for the Supervisor multi-agent pattern in ADK — the orchestrator does not manage sub-agents manually, it just calls them as tools.

```
from google.adk.agents import Agent
from google.adk.tools import AgentTool

# Specialist agent
research_agent = Agent(
    model='gemini-2.0-flash',
    name='research_agent',
    instruction='You are a research specialist. Search and summarize information accurately.'
)

# Wrap it as a tool
research_tool = AgentTool(agent=research_agent)

# Orchestrator uses it
orchestrator = Agent(
```

```
model='gemini-2.0-flash',
name='orchestrator',
instruction='Use research_agent to gather facts, then write a final answer.',
tools=[research_tool]
)
```

How it works: The parent agent calls `research_agent` as if it were a function. ADK runs a full agent session under the hood and returns the sub-agent's final reply as the tool's return value. The parent never sees the sub-agent's internal steps.

7. MCP Tools and OpenAPI Tools

MCP Tools

ADK can connect to any MCP (Model Context Protocol) server and expose all its tools to the agent automatically. This is how you connect to Google's managed MCP servers (BigQuery, Maps, GKE etc.) or any external MCP server.

```
from google.adk.tools.mcp_tool import MCPToolset, SseServerParams

mcp_tools = MCPToolset(
    connection_params=SseServerParams(url='https://mcp.googleapis.com/bigquery')
)

agent = Agent(
    model='gemini-2.0-flash',
    name='data_agent',
    instruction='Use BigQuery tools to answer data questions.',
    tools=[mcp_tools]
)
```

OpenAPI Tools

If a service has a REST API with an OpenAPI (Swagger) spec file, ADK can auto-generate tools from it — no manual function writing needed.

```
from google.adk.tools.openapi_tool import OpenAPIToolset

# ADK reads the spec and creates a tool for each endpoint
api_tools = OpenAPIToolset(spec_url='https://api.example.com/openapi.json')

agent = Agent(
    model='gemini-2.0-flash',
    name='api_agent',
    tools=[api_tools]
)
```

8. Toolsets — Grouping Tools

When an agent has too many tools, the LLM gets confused. ADK solves this with Toolsets — logical groups of related tools that can be managed and provided together.

- A Toolset is a class that implements the `BaseToolset` interface
- It exposes a `get_tools()` method that returns the list of tools in that group

- You can make toolsets dynamic — return different tools based on session state or user role

```
from google.adk.tools import BaseToolset, FunctionTool

class MathToolset(BaseToolset):
    async def get_tools(self, context=None):
        return [
            FunctionTool(func=add_numbers),
            FunctionTool(func=multiply_numbers),
            FunctionTool(func=calculate_percentage),
        ]

agent = Agent(
    model='gemini-2.0-flash',
    name='maths_agent',
    tools=[MathToolset()]
)
```

9. Quick Reference

What you want	How to do it in ADK
Define a custom tool	Write a Python function with type hints + docstring, wrap in FunctionTool
Use Google Search in your agent	Add google_search to the tools list (built-in, no code needed)
Run Python code in your agent	Add code_execution to the tools list (built-in, sandboxed)
Read/write session state in a tool	Add tool_context: ToolContext param, use tool_context.state
Transfer to another agent from a tool	Set tool_context.actions.transfer_to_agent = 'agent_name'
Use another agent as a tool	Wrap with AgentTool(agent=my_agent) and add to tools list
Connect to a BigQuery/Maps MCP server	Use MCPToolset with the Google managed MCP endpoint URL
Auto-generate tools from a REST API	Use OpenAPIToolset(spec_url=...) with the OpenAPI spec
Group tools to avoid overloading context	Implement BaseToolset and return only the relevant tools

Golden rule: The quality of your tool's docstring directly determines how reliably the LLM will use it. Write the docstring as if you are explaining the tool to a junior developer who has never seen your codebase.